

拥塞控制

在不知道路径容量时闭环试探

408 · NET-07

1 本册定位

本 Topic 回答：大量独立发送者共享有限链路与队列、又看不到全局负载时，端点怎样用反馈调节 offered load，避免网络进入高排队、高丢包、低有效吞吐的失控状态？

本册拥有 congestion signal、cwnd、sssthresh、slow start、congestion avoidance、AIMD、fast retransmit/recovery 的 408 经典模型，以及效率、公平、时延和稳定性的权衡。

本册使用 TCP sequence/ACK/connection state，见传输层；可靠性与接收端流控分别不等同于 congestion control。

2 独立阅读词典：拥塞是怎样被观察和控制的

术语	本册中的可操作定义	不要混成什么
congestion / queue	工作量超过某个服务环节能及时处理；queue 是暂存等待的 packet 集合	都不是 packet 自带的标签
offered load λ / service rate C	λ 是 sender 试图注入的速率；C 是瓶颈每秒能处理的量	λ 不是有效吞吐量
cwnd / sssthresh	cwnd 限制在途数据；sssthresh 是从快增转慢增的本地分界估计	都不是路径容量真值
slow start	用 ACK 在 RTT 粒度快速扩大探测窗口	“slow” 不是线性增长
congestion avoidance / AIMD	接近可用范围后慢增；AIMD 是加性增大、乘性减小	不是停止发送或唯一实现
loss/timeout / duplicate ACK ECN/AQM/RED /goodput	丢包、超时和重复 ACK 是不同强度的路径反馈证据 ECN 标记反馈；AQM 提前标记/丢弃；goodput 是交给应用的有效速率	都不是拥塞原因的完整证明 AQM 不是端点 cwnd 更新动作

因此，看到 loss 或 timeout 时，先把它当作路径反馈，再判断 sender 应如何改变 cwnd。丢包是“网络已承受压力”的证据，不是拥塞这个原因本身；重传还可能把 offered load 进一步推高。

3 根本问题：总输入可以超过共享服务能力

设某瓶颈链路服务率为 C ，多个流的总到达速率为 λ ：

- $\lambda < C$ ：队列通常能被清空；
- $\lambda \approx C$ ：小波动就会积累排队；
- $\lambda > C$ 持续存在：队列增长，最终丢包或时延失控。

朴素方案是每个 sender 都尽可能快地发送。单个 sender 看起来在提高自身吞吐，但所有 sender 的独立最优叠加会破坏共享资源：

More offered load $\rightarrow$ Queue growth $\rightarrow$ Delay/loss $\rightarrow$ Retransmission $\rightarrow$ Even more load

拥塞控制必须打断这个正反馈。

4 拥塞是路径状态，不是单个 packet 的属性

发送端通常不知道：瓶颈在哪里、可用容量多少、同时有多少竞争流。它只能观察反馈：

- loss/timeout；
- duplicate ACK pattern；
- increasing RTT/queue delay；
- ECN 等显式标记；
- ACK 到达节奏。

然后更新本地发送约束：

Probe $\rightarrow$ Observe feedback $\rightarrow$ Infer path state $\rightarrow$ Adjust cwnd $\rightarrow$ Probe again

这是反馈控制，不是一次算出带宽后永久使用。

5 cwnd 的对象与单位

cwnd 是 sender 为该 connection 维护的 congestion window，限制网络中允许存在的未确认数据量。TCP 实际发送还受 receiver window：

$$FlightSize \leq \min(cwnd, rwnd).$$

408 题目有时用 MSS 个数表示窗口，有时用 byte。计算前必须写单位；不能把 $cwnd = 8$ 自动解释成 8 byte 或 8 segment。

若 sender 希望充分利用路径，理想在途量与路径 BDP 同量级；大幅超过时，多余数据主要进入 queue，而不是让传播管道变长。

6 Slow Start: 在未知容量下快速探测

连接开始或严重拥塞后，若从 1 MSS 线性增加，找到高速路径容量太慢。Slow Start 借 ACK clock 让 cwnd 在每个 RTT 约翻倍：

```
1 cwnd = 1 MSS
2 one RTT of ACKs -> about 2 MSS
3 next RTT          -> about 4 MSS
4 next RTT          -> about 8 MSS
5 ...
```

名称中的“slow”是相对一开始直接发送巨大 burst；它的增长在 RTT 维度上是指数级。

当 cwnd 到达 ssthresh 附近，切换到 congestion avoidance。ssthresh 不是路径真实容量，而是根据历史拥塞更新的本地分界估计。

7 Congestion Avoidance: 加性试探

接近已知可用范围后，继续指数增长会频繁过冲。经典 congestion avoidance 让 cwnd 每 RTT 大约增加 1 MSS：

$$\Delta cwnd \approx 1 \text{ MSS} / \text{RTT}.$$

每个 ACK 的具体增量可按当前 cwnd 分摊，使一整个窗口被确认后累计约 1 MSS。核心不是背某个实现公式，而是识别增长从 multiplicative 变为 additive。

8 Congestion event 后为什么要乘性下降

若检测到拥塞仍只减一个固定小量，多个 flow 可能长时间把总负载维持在瓶颈之上。经典 AIMD：

Additive Increase + Multiplicative Decrease

用缓慢线性增长持续探测剩余容量，用比例下降快速释放大量在途负载。AIMD 在理想同质条件下还具有向效率与公平状态收敛的几何直觉，但真实 RTT、MSS、应用模式和算法差异会影响公平性。

9 Timeout 与 duplicate ACK 表示不同强度的证据

9.1 Timeout: 反馈闭环严重中断

RTO 到期意味着对应数据在足够长时间内没有获得预期确认。经典 408/Reno 模型把它视为较严重拥塞：更新 ssthresh，将 cwnd 降到较小初值，重新 slow start。

9.2 Three duplicate ACKs: 后续数据仍在通过

连续 duplicate ACK 表明接收端反复看见同一个缺口，但缺口之后的一些 segments 已到达。这通常比 timeout 提供更细的信息：路径仍在交付数据，只是某个 segment 可能丢失。

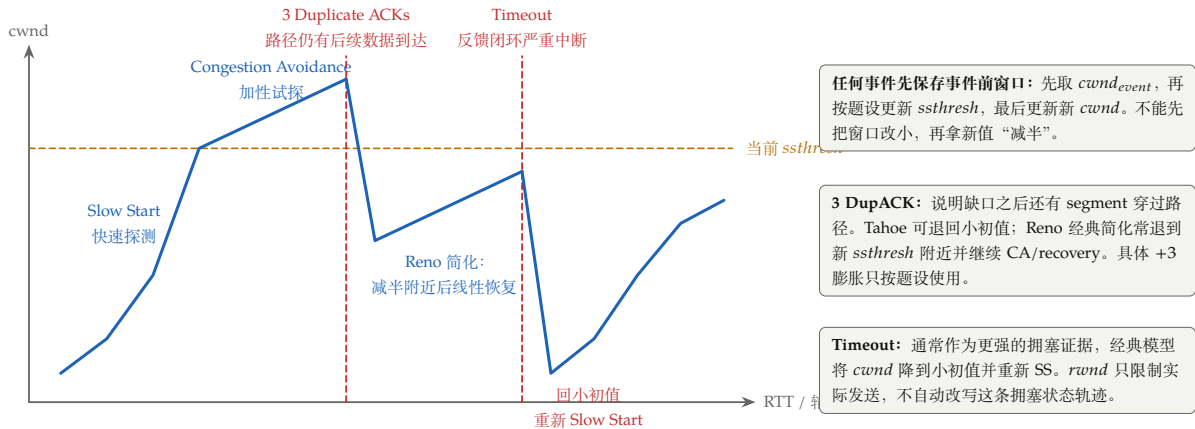
经典模型由此产生：

- fast retransmit: 不等 RTO, 重传推定丢失 segment;
- fast recovery: 不把发送速率退回最初状态, 而按相对温和方式降低并继续。

具体 cwnd/sssthresh 数值更新必须服从题目指定的 Tahoe/Reno 教材模型; 现代 TCP 实现有多种 congestion-control algorithms, 不能把一张 408 状态图冒充所有工程实现。

TCP 经典拥塞窗口: 增长阶段相同, Timeout 与 3 DupACK 的“退让强度”不同

示意采用 408 常见 Reno 简化: Slow Start 约每 RTT 翻倍, CA 约每 RTT +1 MSS; 事件常数必须服从题设。



纵轴只表达窗口状态, 不是路径真实容量; ssthresh 是本地阶段分界估计。现代 TCP 算法并不都沿这条经典 Reno 曲线。

图只固定经典 408 的反馈结构: Slow Start 快增、Congestion Avoidance 加性试探、3 DupACK 的温和退让与 Timeout 的强退让。具体门限、初值与 recovery 临时膨胀必须服从题设版本。

10 ACK clock: 反馈不仅说“收到”, 还提供节奏

当 ACK 随数据穿过瓶颈返回, ACK 到达节奏近似反映路径的交付节奏。sender 按 ACK 释放新数据, 可以避免无节制 burst。

但 ACK compression、delayed ACK、reverse-path congestion 等会扭曲时钟。ACK clock 是有用反馈机制, 不是精确的全局容量测量仪。

11 Queue 的双重身份

Queue 不是纯粹坏事: 短队列吸收 burst, 使链路持续忙碌。问题是持久过量队列:

- latency 增大;
- timeout 和交互性能恶化;
- buffer overflow 导致 loss;
- 新流和短流被已有大队列拖累。

因此优化目标不是“绝不排队”或“永不丢包”, 而是在 utilization、delay、loss、fairness 和 stability 之间取舍。

12 Network-assisted 与 end-to-end feedback

拥塞控制先分“事前约束”与“事后反馈”。事前/open-loop 策略在观察本次拥塞前限制系统行为, 例如接纳控制、资源预留、流量整形、调度与丢弃策略; 反馈/closed-loop 策略检测 queue、delay、loss

或显式标记，再把信号送回能降低 offered load 的主体。前者降低进入危险区的概率，后者适应实际运行状态；二者可以同时存在。

再按反馈 Owner 分层：router/network layer 可调度队列、丢弃或标记 packet，也可在特定体系中反馈显式速率；transport endpoint 根据 ACK/loss/ECN/RTT 等调整发送窗口或速率。网络层“发现/标记拥塞”不自动完成端点减速，端点算法也不能直接观察每台 router 的完整队列真相。

- loss-based control：从丢包/ACK 推断拥塞，部署简单但信号较晚；
- ECN：网络设备在丢包前标记拥塞，端点仍负责降低发送；
- delay-based control：从 RTT/queue delay 变化提前推断，但需要区分路径基线和噪声；
- explicit rate 等更强网络协助模型直接给出速率或资源信息，但需要基础设施支持。

408 核心以经典 TCP loss-based 模型为主；扩展用于解释反馈设计空间，不替代考试指定算法。

因此网络层拥塞与 TCP 拥塞控制不是两种互斥现象：前者描述共享 router/link 的资源状态与网络可提供的控制动作，后者是端点借助有限反馈调节 offered load 的一种闭环实现。

13 408 状态推进：把题设算法写成转移函数

先固定单位 (byte 或 MSS)、初值、‘ssthresh’、ACK 粒度与题设版本。然后对每个事件应用：

事件	进入前状态	经典教材动作	停止/下一分支
new ACK, $cwnd < ssthresh$	slow start	每 ACK 增长；一 RTT 近似翻倍	达阈值转 avoidance
new ACK, $cwnd \geq ssthresh$	avoidance	一 RTT 近似增加 1 MSS	继续探测
timeout	任一发送阶段	$ssthresh \leftarrow$ 题设比例； $cwnd \leftarrow$ 小初值	重传并回 slow start
three duplicate ACKs, Tahoe	检测单缺口	fast retransmit；窗口降到小初值	slow start
three duplicate ACKs, Reno	检测单缺口	fast retransmit；进入/维持 fast recovery	new ACK 后回 avoidance

具体加一、减半、初始窗口和 recovery 膨胀数值以题目定义为准。手册拥有的是“状态、事件、反馈、转移”模型，不把某一版本常数写成所有 TCP 的永恒事实。

轮次与逐 ACK 不能混算

若题目说 slow start 每收到一个确认新数据的 ACK 增加 1 MSS，则一个窗口内的多个 ACK 累计造成下一 RTT 可发送量近似翻倍；不能把“每 ACK 加 1”误写成“每 ACK 翻倍”。题目若直接按 RTT 给窗口序列，则不要再重复逐 ACK 累加。

14 拥塞性能核验层：负载增加为何可能降低吞吐

“资源需求超过可用资源”只是起点。评价拥塞至少要同时观察 offered load、throughput/goodput、queue length、delay/jitter、drop/retransmission。典型负载曲线分为：

区间	曲线关系	系统含义
未饱和	throughput 近似随 offered load 线性增长	新输入大多转化为有效输出
轻度拥塞	输出仍增长，但斜率下降	queue/drop 已开始消耗新增负载
拥塞	offered load 增加，throughput 反而下降	重传、排队与丢弃进入正反馈
死锁极限	goodput 接近 0	大量资源用于无效等待/重复工作
理想控制上界	达容量后近似水平	超额输入在破坏内部状态前被抑制

因此 packet loss 是拥塞的**征兆/反馈载体**，不是拥塞的根本原因；真正原因是持续资源需求与服务能力不匹配。发送量或链路 utilization 变大也不自动等于性能更好，必须把 retransmission/header 从 application goodput 中扣除。

扩大单个 buffer 不能消除 $\lambda > C$ 。它只把早期 drop 改成更长 queue：原 segment 尚在排队时 sender 已因 RTO 重传，原件与副本同时消耗链路，进一步增大 offered load。提高单台 processor 或一条 link 也可能只把瓶颈移到下一资源。正确问题不是“哪个部件不够大”，而是“端到端服务链中哪个约束先饱和，反馈能否在正反馈形成前降低输入”。

15 TCP 算法核验层：逐 ACK 公式与证据强度

slow start 的逐 ACK 教材表达不是无条件加一个完整 MSS。若一个 ACK 新确认了 N bytes，常用上界为

$$\Delta cwnd \leq \min\{N, SMSS\}.$$

当一个窗口的数据都被新 ACK 覆盖时，累积增量约等于原窗口，于是 RTT 粒度近似翻倍。Delayed ACK、ACK loss、application-limited 与 rwnd bottleneck 都可能让实际轨迹偏离理想 1,2,4,8；做题采用轮次近似时，应把这些前提显式写出。

congestion avoidance 若要在一个 RTT 内累计约 1 SMSS，可把每个新 ACK 的增量分摊为量级

$$\Delta cwnd \approx \frac{SMSS^2}{cwnd}.$$

这解释了为什么“每 RTT 加 1 MSS”与“每 ACK 也会更新”并不矛盾。它们是同一过程的不同时间粒度。

三个 duplicate ACK 的证据来自缺口之后仍有 segment 到达：每个失序到达促使 receiver 重复声明同一累计 ACK。它比 timeout 更早，也证明至少还有数据穿过路径；但它仍是启发式阈值，packet reordering 也可能制造 duplicate ACK。阈值取 3 是误判乱序与等待 RTO 之间的工程折中，不是数学必然。

事件	共同动作	Tahoe 教材模型	Reno 简化模型
timeout	先按题设把 $ssthresh$ 更新为事件前窗口的比例	$cwnd$ 回小初值, slow start	同 Tahoe
3 duplicate ACKs	fast retransmit 推定缺口	$cwnd$ 回小初值, slow start	$cwnd \leftarrow ssthresh$, 进入 avoidance/recovery

Reno 的更细实现常在进入 fast recovery 时暂时令 $cwnd = ssthresh + 3 \text{ SMSS}$, 把三个 duplicate ACK 解释为三个 segment 已离开网络; 后续 duplicate ACK 可继续膨胀窗口, 填补缺口的 new ACK 到达后退出 recovery。408 简化题常直接写 $cwnd = ssthresh$ 。两者只可按题设择一, 不能把 recovery 临时膨胀与下一轮稳定窗口重复相加。

16 计算题口径契约：先定义状态时点，再开始算

补充笔记中的细节材料提供了一个内部一致的做题 convention, 但它必须作为题解口径而非协议标准使用:

- “第 n 轮末发生事件”表示第 n 轮仍用事件前窗口发送, 转移动作从第 $n+1$ 轮可见; “当 $cwnd = X$ 时发生事件”则直接在该状态处理。
- 教材对 $cwnd = ssthresh$ 允许 slow start 或 avoidance; 本册默认按 avoidance, 题设另定则覆盖。
- 若题目以整 MSS 做离散轮次, Source 采用 slow start 到门限时封顶、减半向下取整; 真实 byte-based 算法和其他教材可能不采用同一离散化, 必须先声明。
- 事件处理先保存 $cwnd_{event}$, 再算新 $ssthresh$, 最后设置新 $cwnd$ 与阶段; 不能先改窗口再拿新值减半。
- $rwnd$ 只约束实际 send window; 除非题设显式改算法, 拥塞事件仍更新 $cwnd/ssthresh$, 不拿 $\min(cwnd, rwnd)$ 代替事件前 $cwnd$ 。

Reno 全轨迹：两种事件怎样分叉

设 $cwnd_0 = 1 \text{ MSS}$ 、 $ssthresh_0 = 8 \text{ MSS}$; 第 6 轮末发生 3 duplicate ACK, 第 13 轮末 timeout。采用上述 convention:

1, 2, 4, 8, 9, 10, 11 | 5, 6, 7, 8, 9, 10, 11 | 1, 2, 4, 5, 6,

第 6 轮末: $ssthresh = \lfloor 11/2 \rfloor = 5$, $cwnd = 5$,

第 13 轮末: $ssthresh = \lfloor 11/2 \rfloor = 5$, $cwnd = 1$ 。

第一条竖线是 Reno fast recovery 的温和退让, 第二条是 timeout 后回 slow start。若改为 Tahoe, 第一事件也令 $cwnd = 1$, 之后同一轮次的事件前窗口会改变, 后续 $ssthresh$ 不能照抄 Reno。

含恒定 $rwnd = 12$ 、而算法状态 $cwnd = 20$ 时发生 timeout, 实际窗口可只有 12, 但新门限仍按题设用事件前 $cwnd = 20$ 计算为 10, 而不是用 12 得 6。若问累计发送量, 只有在“每轮能填满 $\min(cwnd, rwnd)$ 且 ACK 正常返回”的前提下, 才可直接把各轮窗口求和; 平均 goodput 还要除以总 RTT 时间并扣除重传。

反推曲线可用五个局部形状: 翻倍近似 slow start; 每轮 +1 近似 avoidance; 降到小初值多为 timeout/Tahoe; 减半后线性增长多为 Reno duplicate-ACK 分支; 既不翻倍也不加一的上升, 若恰等

于 *ssthresh*，可能是题目 convention 下的门限封顶。最后用“事件后门限来自事件前窗口”“Reno 简化 recovery 后窗口等于新门限”交叉核验。

17 网络反馈核验层：Tail Drop、AQM 与 RED

拥塞闭环要区分 signal generation 与 endpoint response。router 决定何时 drop/mark、影响哪些 flows；TCP sender 决定如何改 *cwnd*。只写“路由器丢包”没有说明信号质量。

FIFO tail drop 在 queue 满后连续丢弃新到 packet，容易同时打击共享队列中的许多 TCP flows。这些 flows 近乎同时降低窗口、随后又同步增长，形成 global synchronization，使链路负载大幅振荡。扩大 queue 可能推迟这一刻，却增加 standing delay 和 timeout 风险。

AQM 在 queue 真正满之前主动 drop 或 ECN-mark 少量 packet，目标是让反馈**更早、更分散**。RED 是教学中的一个实例：

平均队列长度 $\bar{q}$	动作	设计目的
$\bar{q} < q_{min}$	正常入队	吸收短 burst
$q_{min} \leq \bar{q} < q_{max}$	以随 $\bar{q}$ 增长的概率 drop/mark	随机分散反馈，避免 flows 同步
$\bar{q} \geq q_{max}$	drop/mark 强制动作	阻止队列继续失控

判据使用 average queue 而非瞬时 queue，相当于过滤短 burst；中间区随机选择而非确定性全丢，是为了不把提前门限重新变成另一种 tail drop。RED 的具体参数很难调，不能从这个经典例子推出它是唯一或当前通用 AQM；稳定 Owner 是“主动队列管理改善反馈时点与分散度”的机制。

open-loop 与 closed-loop 也不是互斥协议。Admission control、reservation、traffic shaping、scheduler 等在运行事件前限制风险；closed-loop 执行 monitor → convey signal → adjust。抑制分组本身会增加负载，捎带 ECN 标记开销小但需端点配合，探测则有固定开销与时延。反馈太快可能振荡，太慢则来不及阻止正反馈，控制时间常数本身就是系统设计变量。

18 概念边界

概念 A	≠ 概念 B	真正区别与题目信号	混淆后果
Congestion	≠ Receiver overflow	path resources vs receiver buffer	<i>cwnd/rwnd</i> 更新写反
<i>cwnd</i>	≠ <i>rwnd</i>	sender inference vs receiver advertisement	状态 Owner 错
<i>ssthresh</i>	≠ Bottleneck capacity	阶段阈值 vs 外部动态容量	把阈值当带宽
Timeout	≠ Certain congestion	强推断，仍可能是误码/长时延	把观测当事实
Duplicate ACK	≠ New-data ACK	重复缺口 vs 推进连续前缀	ACK 计数错误
Flow rate	≠ Goodput	可含首部/重传 vs 应用有效输出	发送更多即更好
Slow start	≠ Linear growth	低起点、RTT 维度近似指数增长	轮次推演错误
Classic Reno	≠ All modern TCP	408 算法实例 vs 多种现实实现	旧图当唯一事实
AQM	≠ Endpoint response	router 发信号 vs sender 调窗口	跨层动作合成一步

19 做题调用协议

1. 标出 *cwnd*、*ssthresh*、*rwnd* 的单位和初值；
2. 画 RTT 轮次或逐 ACK 事件，按题目粒度更新；
3. 判断当前处于 slow start、avoidance 还是 recovery；
4. 区分 timeout 与 duplicate ACK 触发的教材分支；
5. 计算实际发送上限时取 $\min(cwnd, rwnd)$ 并扣除 in-flight；
6. 明确题目采用 Tahoe、Reno 或自定义规则；
7. 用 BDP、queue 和 goodput 检查窗口变化是否有物理意义。

20 贯穿母例：为什么 1、2、4、8 后不能永远翻倍

假设瓶颈在途与队列可暂时容纳约 10 MSS。sender 从 1 MSS slow start：

```

1 1 -> 2 -> 4 -> 8: path may absorb
2 8 -> 16: offered flight overshoots available capacity
3 -> queue grows / loss signal appears
4 -> sender reduces cwnd and records ssthresh
5 -> later probes additively near the observed boundary

```

Slow start 解决“完全不知道容量时怎样快速接近”；AIMD 解决“接近边界后怎样持续试探并在过冲时退让”。二者不是两套互不相关的背诵规则，而是探索阶段不同。

21 高频 First Divergence

- 把 cwnd 当 receiver 通告值：状态 owner 错；
- 每收到一个 ACK 就让 cwnd 翻倍：混淆逐 ACK 增量与逐 RTT 效果；
- 到 ssthresh 后停止增长：忘记 avoidance 仍在探测；
- timeout 与三次重复 ACK 使用同一分支：丢失证据强度分层；
- 只算发送速率不算 goodput：重传负载被当成有效数据；
- 未读题设就套 Reno：没有确认教材模型。

22 一页压缩与复原问题

Probe capacity → Read ACK/loss/ECN/delay → Infer congestion → Adjust cwnd → Probe again

1. 为什么 congestion control 必须是闭环而不是固定速率？
2. Slow Start 为什么既“慢”又近似指数增长？
3. AIMD 的加性与乘性分别解决什么？
4. duplicate ACK 为什么通常比 timeout 提供更细的路径信息？
5. Queue 在什么范围内有益，何时开始伤害系统？

23 来源与校正说明

- 早期归档没有独立拥塞控制正文；本轮已核验用户提供的新提供的中文补充笔记，而非从术语表复制；
- transport/tcp-congestion.md 已吸收负载曲线、拥塞推断、逐 ACK 增长、Tahoe/Reno、fast retransmit/recovery 与 AIMD 设计理由；
- transport/tcp-congestion-detail.md 已吸收事件时点、阈值等号、离散封顶/取整 convention、逐轮/反推/累计量自检；这些口径被明确限制为题解契约；
- network/network-layer-functions.md 中 congestion、open/closed loop、tail drop、global synchronization、AQM/RED 部分归本册；route/forwarding 内容仍由 NET04/NET05 拥有；
- cwnd、rwnd 共同限制在途数据，以及 slow start/congestion avoidance/fast retransmit/recovery 的经典边界依据 RFC 5681 校正；
- 具体窗口初值和现代算法会演进，本册将工程现实与 408 指定模型分层，不把版本相关常数写成永恒机制。